

SISTEMA DE ACHADOS E PERDIDOS

Divisão individual de tarefas

Projeto de laboratório de programação em Python

Grupo	3 integrantes
Prazo final	08/10/2026
Aplicação	Sistema em terminal
Finalidade	Cadastrar, pesquisar e devolver objetos perdidos ou encontrados

REGRAS DO TRABALHO

Cada integrante desenvolve e testa sua própria parte. Antes da entrega, todos devem integrar os módulos, revisar o projeto completo e saber explicar onde listas, dicionários, tuplas, for, while, if, funções, módulos, exceções, ordenação, strings e match/case foram utilizados.

PESSOA 1

Cadastro e manutenção dos objetos

Nome do integrante: _____

OBJETIVO

Criar e manter os registros de objetos perdidos e encontrados, garantindo que cada item tenha código único e informações válidas.

Arquivos sob responsabilidade

`objetos.py`

Funções previstas

`gerar_codigo()` | `cadastrar_objeto()` | `listar_objetos()` | `localizar_por_codigo()` | `atualizar_objeto()` | `remover_objeto()`

Tarefas individuais

- ☐ Definir a estrutura padrão do dicionário de cada objeto.
- ☐ Cadastrar objetos perdidos e encontrados.
- ☐ Gerar automaticamente um código único para cada registro.
- ☐ Validar nome, categoria, descrição, local, data, tipo e responsável.
- ☐ Listar todos os objetos de forma organizada.
- ☐ Localizar um objeto pelo código para uso pelos outros módulos.
- ☐ Permitir a atualização das informações de um objeto.
- ☐ Permitir a remoção de um registro após confirmação.
- ☐ Receber a lista de objetos por parâmetro, evitando variáveis globais.

Conteúdos de Python utilizados

Conteúdo	Como será utilizado
Listas	Armazenar todos os objetos cadastrados.
Dicionários	Representar cada objeto e seus dados.
Tuplas	Representar datas e categorias disponíveis.
Strings	Tratar nomes, descrições, locais e responsáveis.
for e if	Percorrer registros e validar condições.
Funções	Separar cadastro, listagem, atualização e remoção.

Testes que devem ser feitos

- Cadastrar um objeto com nome vazio.
- Cadastrar objetos com nomes iguais e confirmar códigos diferentes.
- Atualizar e remover um código inexistente.
- Listar quando não houver registros.
- Verificar se o código permanece único depois de uma remoção.

CRITÉRIO DE CONCLUSÃO

A parte estará concluída quando os objetos puderem ser cadastrados, listados, localizados, alterados e removidos sem depender do menu principal.

PESSOA 2

Pesquisas, devoluções e validações

Nome do integrante: _____

OBJETIVO

Implementar as pesquisas do sistema, registrar devoluções e centralizar o tratamento das entradas inválidas.

Arquivos sob responsabilidade

`buscas.py` | `devolucoes.py` | `validacoes.py`

Funções previstas

`buscar_por_codigo()` | `buscar_por_nome()` | `buscar_por_categoria()` | `buscar_por_local()` | `registrar_devolucao()` | `validar_data()` | `ler_inteiro()`

Tarefas individuais

- ☐ Pesquisar objeto por código.
- ☐ Pesquisar por nome usando busca parcial e sem diferenciar maiúsculas de minúsculas.
- ☐ Pesquisar por categoria, local, tipo e status.
- ☐ Exibir todos os resultados encontrados na pesquisa.
- ☐ Registrar o nome de quem recebeu o objeto e a data da devolução.
- ☐ Alterar o status do objeto para Devolvido.
- ☐ Impedir a devolução de um objeto já devolvido.
- ☐ Criar validações reutilizáveis para números, textos vazios e datas.
- ☐ Usar try/except para impedir que entradas inválidas encerrem o programa.

Conteúdos de Python utilizados

Conteúdo	Como será utilizado
for	Percorrer objetos durante as pesquisas.
if	Verificar correspondências e regras de devolução.
Strings	Normalizar termos com <code>strip()</code> e <code>lower()</code> .
Tuplas	Representar a data da devolução.
Dicionários	Registrar os dados da devolução.
Exceções	Tratar códigos, datas e opções inválidas.

Testes que devem ser feitos

- Pesquisar por um objeto inexistente.
- Pesquisar utilizando letras maiúsculas e minúsculas.
- Digitar letras no lugar do código.
- Informar uma data inválida.
- Devolver um objeto inexistente.
- Tentar devolver duas vezes o mesmo objeto.

CRITÉRIO DE CONCLUSÃO

A parte estará concluída quando as pesquisas retornarem os registros corretos e uma devolução válida atualizar o status sem aceitar dados incorretos ou duplicidade.

PESSOA 3

Menu, ordenação, relatórios e integração

Nome do integrante: _____

OBJETIVO

Criar a navegação principal, ordenar os registros, apresentar estatísticas e conectar todos os módulos do sistema.

Arquivos sob responsabilidade

main.py | relatorios.py

Funções previstas

exibir_menu() | executar_sistema() | ordenar_objetos() | mostrar_estatisticas() | mostrar_por_categoria()

Tarefas individuais

- ☐ Criar as listas principais de objetos e devoluções.
- ☐ Montar o menu principal e mantê-lo ativo com while.
- ☐ Usar match/case para controlar todas as opções.
- ☐ Importar corretamente as funções dos outros módulos.
- ☐ Permitir ordenar objetos por nome, data, categoria, local ou status.
- ☐ Criar estatísticas de objetos perdidos, encontrados, disponíveis e devolvidos.
- ☐ Mostrar a quantidade de objetos por categoria.
- ☐ Tratar opções inexistentes no menu.
- ☐ Executar e testar o fluxo completo do sistema.
- ☐ Coordenar a integração final e resolver incompatibilidades entre os módulos.

Conteúdos de Python utilizados

Conteúdo	Como será utilizado
while	Manter o menu funcionando até a opção de saída.
match/case	Direcionar cada opção do menu.
Ordenação	Organizar os objetos conforme o campo escolhido.
Dicionários	Contabilizar categorias e situações.
Módulos	Integrar objetos, buscas, devoluções e relatórios.
Exceções	Evitar encerramento por entradas incorretas.

Testes que devem ser feitos

- Escolher uma opção inexistente.
- Digitar letras quando o menu espera número.
- Ordenar uma lista vazia.
- Ordenar por nome, data e categoria.
- Conferir estatísticas antes e depois de uma devolução.
- Realizar o fluxo completo usando funções importadas.

CRITÉRIO DE CONCLUSÃO

A parte estará concluída quando o usuário puder acessar todas as funcionalidades pelo menu e os relatórios apresentarem dados corretos após a integração.

Acordos para integração

ESTRUTURA PADRÃO DO OBJETO

Todos deverão usar as mesmas chaves: codigo, nome, categoria, descricao, local, data, tipo, status e responsavel.

Regras definidas pelo grupo

- Objeto perdido inicia com status Procurando.
- Objeto encontrado inicia com status Disponível.
- Depois da entrega, o status passa para Devolvido.
- O código é gerado automaticamente e nunca deve ser repetido.
- As datas serão representadas por tuplas no formato (dia, mês, ano).
- As funções receberão as listas necessárias por parâmetro.

Cronograma

- 24/09: definir estrutura dos dados, regras e nomes das funções.
- 25/09 a 28/09: desenvolvimento individual.
- 29/09: primeira integração dos módulos.
- 30/09 a 02/10: finalizar funcionalidades e exceções.
- 03/10 e 04/10: testes gerais e correção de erros.
- 05/10 e 06/10: revisão coletiva e troca de testes.
- 07/10: simulação da apresentação.
- 08/10: teste final e entrega.

Checklist final do grupo

- ☐ Todos os módulos são importados sem erro.
- ☐ O sistema utiliza todos os conteúdos exigidos.
- ☐ Entradas inválidas não encerram o programa.
- ☐ As buscas funcionam com termos parciais.
- ☐ Uma devolução atualiza corretamente o status.
- ☐ A ordenação e as estatísticas apresentam resultados corretos.
- ☐ Os três integrantes sabem explicar o sistema completo.

IMPORTANTE

Não deixar a integração para o último dia. Cada pessoa deve enviar sua parte funcionando e com os nomes de funções e chaves exatamente iguais aos definidos neste documento.